TrainSel: Optimal Training Set Selection and Mixed-Integer Optimization for Plant Breeding

Deniz Akdemir Julio Isidro Sanchez Simon Rio Javier Fernández-González

2026-May-17

Abstract

`TrainSel` solves mixed-integer optimization problems that arise routinely in plant breeding and quantitative genetics, most prominently, the selection of optimal training sets for genomic prediction. The package takes a flexible, user-defined objective function and a description of the candidate sets, and returns the subset (and optional continuous parameters) that maximizes that objective. This vignette walks through (i) classical training-set design criteria (D-optimality, CDmean, Maximin, PEV, Minimax) for single and multiple environments, with and without an explicit experimental design; (ii) multi-objective optimization and Pareto fronts; (iii) applications beyond training-set design, including anchor-marker selection, optimal parental proportions, and optimal genomic mating; and (iv) practical guidance for tuning the algorithm's hyperparameters and estimating computation time.

Contents

1. Introduction

1.1 What `TrainSel` does

`TrainSel` is an R package for **mixed-integer optimization** problems that appear in plant breeding, quantitative genetics, and applied statistics. Its canonical use case is **training set selection for genomic prediction**: out of a large pool of selection candidates with marker data, choose a smaller subset to phenotype so that the resulting genomic prediction model generalizes well to the genotypes one really cares about.

More generally, `TrainSel` searches for the subset (and, optionally, the continuous parameters) that **maximizes a user-supplied objective function**. Because the user writes the objective, the package can be repurposed for problems that look very different on the surface, variable selection, optimal mating allocation, parental proportions, and even general-purpose function optimization, without any change to the underlying engine.

Practically, every `TrainSel` call has the same shape:

Argument	What you supply
`Data`	Anything your objective needs (relationship matrix, markers, …)
`Candidates`	One or more vectors of integer IDs (or `c(lower, upper)` bounds for continuous variables)
`setsizes`	How many items to draw from each candidate vector
`settypes`	Whether each set is ordered/unordered, with/without repetition, or continuous
`Stat`	A function returning the value to **maximize**
`control`	Algorithm settings (built with `SetControlDefault`)

The `settypes` codes will appear throughout the examples, so it is worth keeping them in mind:

Code	Meaning
`UOS`	**U**n-**O**rdered **S**et, order does not matter; no repeats
`OS`	**O**rdered **S**et, order matters; no repeats
`UOMS`	**U**n-**O**rdered **M**ulti-**S**et, order does not matter; repeats allowed
`OMS`	**O**rdered **M**ulti-**S**et, order matters; repeats allowed
`BOOL`	Binary selection (include / exclude each candidate)
`DBL`	Continuous variable bounded by `c(lower, upper)`

The maximization output is returned in a single list with, most importantly — `BestSol_int` (the selected integers), `BestSol_DBL` (the selected continuous values), `BestVal` (the achieved objective), and `maxvec` (the trajectory of the best objective over iterations, useful for assessing convergence).

1.2 The wheat example dataset

All examples in this vignette use the `WheatData` object shipped with the package. The data come from the public Triticeae Toolbox (https://triticeaetoolbox.org/) and consist of:

- `Wheat.M`: a 200×4670 marker matrix coded as $-1/0/1$.
- `Wheat.K`: a 200×200 genomic relationship matrix derived from the markers.
- `Wheat.Y`: a (simulated) plant-height phenotype for the 200 wheat lines, generated under an **infinitesimal model**.

```
library(TrainSel)
data("WheatData")

Wheat.M[1:5, 1:5]
```

```
##       IWA1 IWA2 IWA3 IWA4 IWA5
## Line1    1    1   -1   -1   -1
## Line2    1    1    1   -1   -1
## Line3    1    1    1   -1   -1
## Line4    1    1    1    1   -1
## Line5    1    1    1   -1   -1
```

```
Wheat.K[1:5, 1:5]
```

```
##            Line1      Line2      Line3      Line4      Line5
## Line1  1.9578361  0.8261588  0.7546713  0.5149389 -0.2697654
## Line2  0.8261588  2.0033376  0.5756170  0.5241991 -0.2315324
## Line3  0.7546713  0.5756170  1.9807687  0.7077443 -0.2888690
## Line4  0.5149389  0.5241991  0.7077443  2.2816612 -0.2645939
## Line5 -0.2697654 -0.2315324 -0.2888690 -0.2645939  1.8086996
```

```
Wheat.Y[1:5, ]
```

```
##         id plant.height
## 1846 Line1     138.4471
## 250  Line2     122.5955
## 1541 Line3     134.7883
## 1516 Line4     121.4741
## 508  Line5     127.3920
```

Throughout the rest of the vignette we will repeatedly assume the following breeding scenario:

> Out of 200 genotyped wheat lines, only the first **160** are physically available to be planted in a phenotyping trial; the remaining **40** (lines 161–200) are unavailable for phenotyping but we still

have markers for them. Phenotyping is costly, so we can field at most **50** of the 160 candidates. Our goal is to pick which 50 to phenotype so that, after training a genomic prediction model on those 50, the predictions for unphenotyped lines are as good as possible.

2. Selecting a training set in a single environment

We start with the simplest setting: a single homogeneous environment, no explicit design matrix, and a training-set size fixed at 50.

A useful conceptual distinction throughout the package is whether or not we have a **specific target set** of genotypes we want to predict:

- **Un-targeted optimization** maximizes a property of the training set itself (e.g. how well it spans the genetic space) without privileging any specific prediction set.
- **Targeted optimization** maximizes prediction quality for a known target set , typically the genotypes for which we want genomic estimated breeding values (GEBVs).

Not every criterion is sensitive to the target/un-target distinction; for those that are, both forms are illustrated below.

2.1 Un-targeted criteria

`TrainSel` ships with great flexibility, which means you typically write the objective function yourself. The four functions below cover the criteria most commonly used in the literature; they are short enough to serve as templates for your own variants.

2.1.1 D-optimality

D-optimality is a model-based criterion derived from a linear model

$$y = \mathbf{1}\mu + f(M)\beta + \epsilon,$$

where y is the n-vector of phenotypes, M is the $n \times m$ marker matrix of the n training genotypes, $f(M)$ is an $n \times q$ feature matrix (commonly the leading q principal components of M), and the residual ϵ is i.i.d. $\mathcal{N}(0, \sigma_e^2)$. Under this model the covariance of $\hat{\beta}$ is proportional to $[f(M)'f(M)]^{-1}$, so a D-optimal training set minimizes the determinant of that inverse — equivalently, it **maximizes** $\log\det(f(M)'f(M))$. For the determinant to be defined we need $q < n$.

We use the first 30 principal components of the centered marker matrix as our feature matrix.

```
# Center markers and extract 30 principal components
Wheat.M_centered <- scale(Wheat.M, center = TRUE, scale = FALSE)
svdWheat.M_centered <- svd(Wheat.M_centered, nu = 30, nv = 30)
PC <- Wheat.M_centered %*% svdWheat.M_centered$v
dim(PC)
```

```
## [1] 200  30
```

```
# Data object for the optimizer, a list with whatever Stat() needs
dataDopt <- list(FeatureMat = PC)

# Objective: log-determinant of f(M)'f(M) restricted to the selected rows
DOPT <- function(soln, Data) {
  Fmat <- Data[["FeatureMat"]]
  determinant(crossprod(Fmat[soln, ]), logarithm = TRUE)$modulus
}
```

Algorithm parameters are bundled into a `TrainSel_Control` object. For every example in this vignette we use the **demo** preset so that the code can be run without a license key. Tuning these parameters is covered in Section 6.

```
# Demo-compatible control object
TSC <- SetControlDefault(size = "demo", complexity = "low_complexity")

# The control object is a plain list, so any element can be tweaked.
# Below we show this, but in general we do NOT recommend hand-editing it,
# since deviating from the demo presets will exceed demo limits.
TSC$niterations
```

```
## [1] 100
```

```
TSC$niterations <- 90
```

We now run the optimizer. `Candidates = list(1:160)` says the integers we may pick from come from the candidate pool of 160 lines; `setsizes = c(50)` asks for 50 of them; `settypes = "UOS"` declares that the selected set is unordered with no repetition.

```
TSOUTD <- TrainSel(
  Data       = dataDopt,
  Candidates = list(1:160),
  setsizes   = c(50),
  settypes   = "UOS",
  Stat       = DOPT,
  control    = TSC
)
```

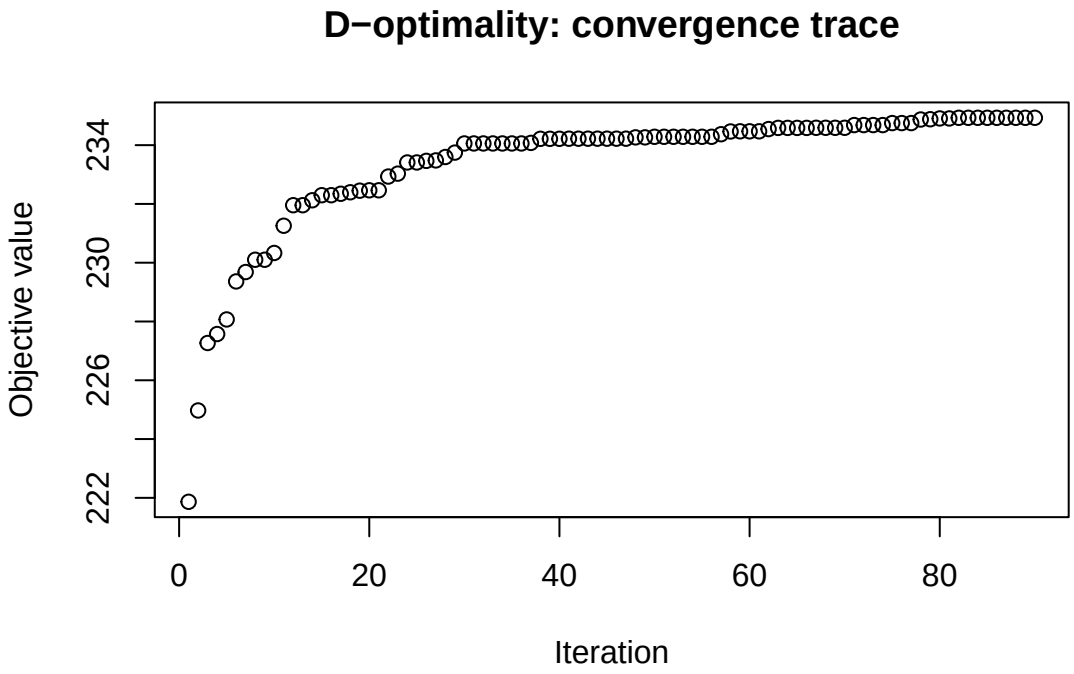
```
## [1] "Using demo version."
## Limited to selecting 100 integer and 3 double solutions with hyperparameters in SetControlDefault(si
## [1] "If you are interested in upgrading to full version, please contact us at j.isidro@upm.es"
## Maximum number of iterations reached.
```

```
# Map the integer indices back to wheat line names
head(rownames(Wheat.M)[TSOUTD$BestSol_int])
```

```
## [1] "Line4"  "Line7"  "Line13" "Line15" "Line16" "Line17"
```

Checking convergence. Always inspect the trajectory of the objective. If `maxvec` is still climbing at the end of the run, you have not converged and should increase `niterations` (or upgrade from the demo size).

```
plot(TSOUTD$maxvec,
     xlab = "Iteration",
     ylab = "Objective value",
     main = "D-optimality: convergence trace")
```



2.1.2 CDmean (un-targeted)

CDmean is a **G-BLUP–based** criterion. Starting from

$$y = \mathbf{1}\mu + Zu + \epsilon, \qquad u \sim \mathcal{N}(0, \sigma_g^2 G), \quad \epsilon \sim \mathcal{N}(0, \sigma_e^2 I),$$

with G the relationship matrix, the coefficient-of-determination matrix of $\hat{u}$ is

$$\mathrm{CD} = (GZ'PZG) \oslash G,$$

with $P = V^{-1} - V^{-1}\mathbf{1}(\mathbf{1}'V^{-1}\mathbf{1})^{-1}\mathbf{1}'V^{-1}$ the projection that removes the intercept and $\oslash$ element-wise division. Each diagonal entry of CD is a per-genotype reliability between 0 and 1; **CDmean averages these diagonals**, and larger is better.[1]

In the un-targeted form, we average the CD over **the genotypes not in the training set**, since those are the genotypes the trained model will actually have to predict.

```
# Use only the 160 candidates' block of K (this is the "un-targeted" version)
dataCDMEANopt <- list(G = Wheat.K[1:160, 1:160], lambda = 1)

CDMEANOPT <- function(soln, Data) {
  G      <- Data[["G"]]
  lambda <- Data[["lambda"]]
  Vinv   <- solve(G[soln, soln] + lambda * diag(length(soln)))
  outmat <- (G[, soln] %*% (Vinv - (Vinv %*% Vinv) / sum(Vinv)) %*% G[soln, ]) / G
  # Mean CD over genotypes NOT in the selected training set
  mean(diag(outmat[-soln, -soln]))
}

TSOUTCD <- TrainSel(
  Data       = dataCDMEANopt,
```

[1] A risk-averse variant maximizes the **minimum** diagonal element (`CDmin`) instead of the mean.

```
  Candidates = list(1:160),
  setsizes   = c(50),
  settypes   = "UOS",
  Stat       = CDMEANOPT,
  control    = TSC,
  Verbose    = FALSE
)
```

```
## Maximum number of iterations reached.
```

```
head(rownames(Wheat.M)[TSOUTCD$BestSol_int])
```

```
## [1] "Line4"  "Line6"  "Line7"  "Line9"  "Line11" "Line15"
```

2.1.3 Maximin distance (space-filling)

A non-parametric, model-free alternative is to select training lines that are as *spread out* as possible in marker space. The **Maximin** criterion maximizes the minimum pairwise distance among training genotypes, producing a space-filling design.[2]

```
dataMaximin <- list(DistMat = as.matrix(dist(Wheat.M_centered)))

MaximinOPT <- function(soln, Data) {
  Dsoln <- Data[["DistMat"]][soln, soln]
  # Lower triangle of pairwise distances within the selected set
  min(Dsoln[lower.tri(Dsoln, diag = FALSE)])
}

TSOUTMaximin <- TrainSel(
  Data       = dataMaximin,
  Candidates = list(1:160),
  setsizes   = c(50),
  settypes   = "UOS",
  Stat       = MaximinOPT,
  control    = TSC,
  Verbose    = FALSE
)
```

```
## Maximum number of iterations reached.
```

```
head(rownames(Wheat.M)[TSOUTMaximin$BestSol_int])
```

```
## [1] "Line4"  "Line11" "Line16" "Line17" "Line18" "Line19"
```

2.2 Targeted criteria

When the genotypes we want to predict are known in advance, for example, the 40 lines (161–200) that we cannot phenotype, we can shape the objective so that the training set is good *for those targets specifically.*

2.2.1 Mean PEV (linear-model based)

The **Mean Prediction Error Variance (Mean PEV)** is the linear-model analogue of D-optimality but referenced to a known target set:

$$\text{Mean PEV} = \text{tr}[F_T(F_S'F_S)^{-1}F_T']/|T|,$$

[2] A close cousin, the **Maximean** criterion, maximizes the mean pairwise distance instead.

where F_S are the rows of $f(M)$ for the selected training genotypes and F_T those for the target genotypes. We **minimize** Mean PEV; equivalently we maximize its negative. (The code below returns the mean of the diagonal of the inner matrix; `TrainSel` will maximize this, so when adapting this template to a problem where you want PEV minimized, return `-mean(diag(...))`.)

```
dataPEVlm <- list(FeatureMat = PC, Target = 161:200)

PEVlmOPT <- function(soln, Data) {
  Fmat <- Data[["FeatureMat"]]
  targ <- Data[["Target"]]
  mean(diag(Fmat[targ, ] %*% solve(crossprod(Fmat[soln, ])) %*% t(Fmat[targ, ])))
}

TSOUTPEVlm <- TrainSel(
  Data       = dataPEVlm,
  Candidates = list(1:160),
  setsizes   = c(50),
  settypes   = "UOS",
  Stat       = PEVlmOPT,
  control    = TSC,
  Verbose    = FALSE
)
```

```
## Maximum number of iterations reached.
```

```
head(rownames(Wheat.M)[TSOUTPEVlm$BestSol_int])
```

```
## [1] "Line11" "Line12" "Line17" "Line32" "Line33" "Line35"
```

2.2.2 Targeted CDmean

The targeted version of CDmean is almost identical to its un-targeted sibling; the only change is that the diagonals being averaged are now those of the *target* genotypes rather than of the non-training candidates.

```
# Note: G uses the full 200x200 matrix because the target rows live in there
dataCDMEANTargetOpt <- list(G = Wheat.K, lambda = 1, Target = 161:200)

CDMEANOPTTarget <- function(soln, Data) {
  G      <- Data[["G"]]
  lambda <- Data[["lambda"]]
  targ   <- Data[["Target"]]
  Vinv   <- solve(G[soln, soln] + lambda * diag(length(soln)))
  outmat <- (G[, soln] %*% (Vinv - (Vinv %*% Vinv) / sum(Vinv)) %*% G[soln, ]) / G
  mean(diag(outmat[targ, targ]))
}

TSOUTCDTarg <- TrainSel(
  Data       = dataCDMEANTargetOpt,
  Candidates = list(1:160),
  setsizes   = c(50),
  settypes   = "UOS",
  Stat       = CDMEANOPTTarget,
  control    = TSC,
  Verbose    = FALSE
)
```

```
## Maximum number of iterations reached.
```

```r
head(rownames(Wheat.M)[TSOUTCDTarg$BestSol_int])
```

```
## [1] "Line8"  "Line13" "Line17" "Line21" "Line22" "Line23"
```

2.2.3 Minimax distance

The distance-based counterpart to PEV is **Minimax**: minimize the maximum distance from any target genotype to its nearest training genotype. In words: keep targets *covered*, make sure no target is genetically far away from at least one training line. We negate the maximum because `TrainSel` maximizes.

```r
dataMiniMax <- list(DistMat = as.matrix(dist(Wheat.M_centered)))

MiniMaxOPT <- function(soln, Data) {
  # Distances from each training genotype to each target genotype (161:200)
  Dsoln <- Data[["DistMat"]][soln, 161:200]
  -max(c(unlist(Dsoln)))
}

TSOUTMinimax <- TrainSel(
  Data       = dataMiniMax,
  Candidates = list(1:160),
  setsizes   = c(50),
  settypes   = "UOS",
  Stat       = MiniMaxOPT,
  control    = TSC,
  Verbose    = FALSE
)
```

```
## Maximum number of iterations reached.
```

```r
head(rownames(Wheat.M)[TSOUTMinimax$BestSol_int])
```

```
## [1] "Line5"  "Line6"  "Line8"  "Line11" "Line13" "Line15"
```

2.3 Multiple objectives at once

Real breeding decisions are rarely about a single number. `TrainSel` supports **multi-objective optimization** through Pareto-frontier search: instead of a single best solution it returns a set of non-dominated solutions, each one representing a different trade-off between the objectives.

Below we simultaneously (i) spread the training set in marker space (maximize mean pairwise distance among training lines) and (ii) cover the target set (minimize the mean distance from training lines to targets, equivalently, maximize its negative). The objective function returns a length-2 numeric vector, and `nStat = 2` tells `TrainSel` to treat both entries as objectives to maximize.

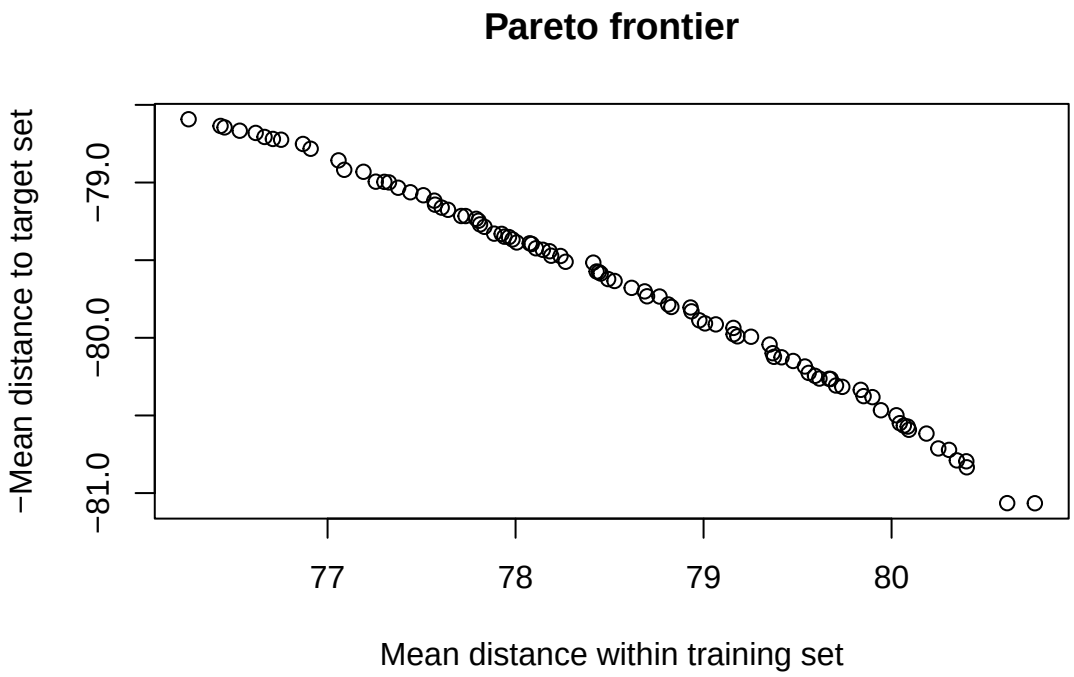
```r
dataMultOpt <- list(DistMat = as.matrix(dist(Wheat.M_centered)))

MultOPT <- function(soln, Data) {
  D <- Data[["DistMat"]]
  # Objective 1: mean within-training distance (spread)
  withinTrain <- mean(D[soln, soln])
  # Objective 2: -mean distance to targets (proximity to target set)
  toTarget    <- -mean(D[soln, 161:200])
  c(withinTrain, toTarget)
}

TSOUTMultOPt <- TrainSel(
```

```
  Data       = dataMultOpt,
  Candidates = list(1:160),
  setsizes   = c(50),
  settypes   = "UOS",
  Stat       = MultOPT,
  nStat      = 2,
  control    = TSC,
  Verbose    = FALSE
)
```

```
## Maximum number of iterations reached.
```

The columns of `BestVal` are the (obj1, obj2) coordinates of the Pareto frontier; plotting them shows the trade-off explicitly.

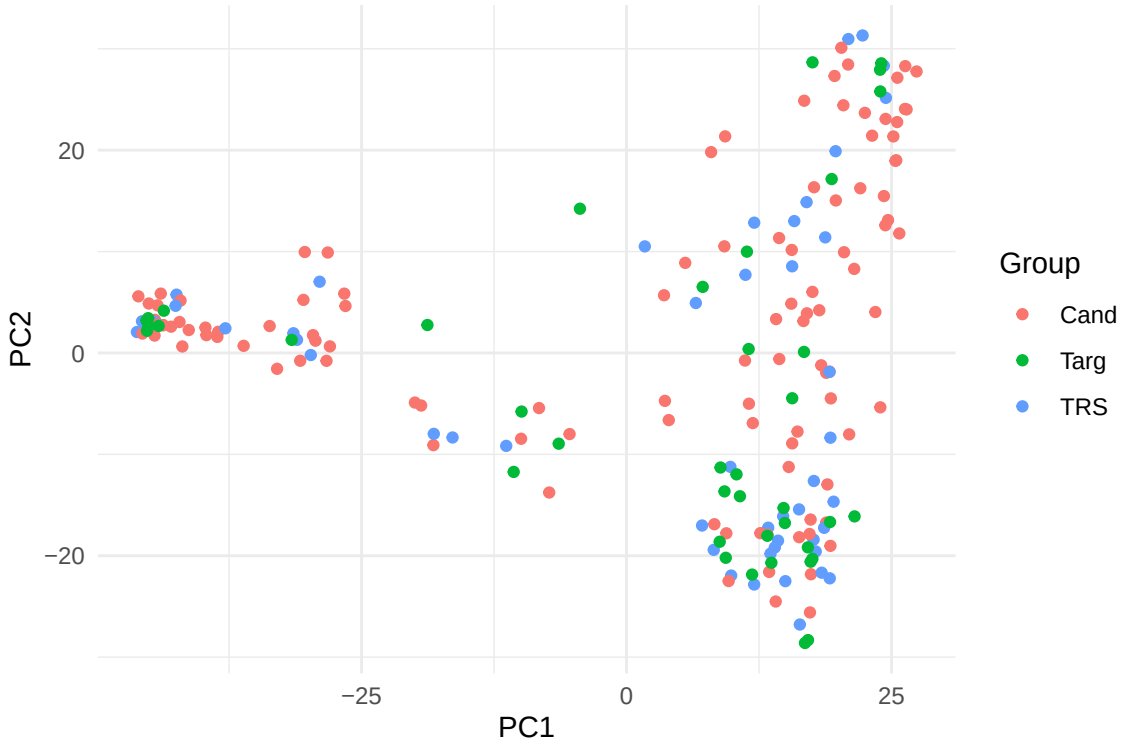
```
FrontierSols <- t(TSOUTMultOPt$BestVal)
plot(FrontierSols,
     xlab = "Mean distance within training set",
     ylab = "-Mean distance to target set",
     main = "Pareto frontier")
```



A practitioner can now pick a frontier point that matches their preferred trade-off and inspect the corresponding solution in PC space:

```
# Pick a solution in the upper-right region of the frontier
Solns    <- which((FrontierSols[, 1] >= 79) & (FrontierSols[, 2] >= -80))
Selected <- TSOUTMultOPt$BestSol_int[, Solns[1]]

Groups            <- rep("Cand", 200)
Groups[161:200]   <- "Targ"
Groups[Selected]  <- "TRS"

PlotData <- data.frame(x1 = PC[, 1], x2 = PC[, 2],
                       Groups = factor(Groups))
```

```r
library(ggplot2)
ggplot(PlotData, aes(x = x1, y = x2, color = Groups)) +
  geom_point() +
  labs(x = "PC1", y = "PC2", color = "Group") +
  theme_minimal()
```



3. Adding an experimental design

So far we treated the trial as a single homogeneous environment. In practice a trial usually has a **blocking structure** (rows, columns, replicates, checks…) and we want to estimate genomic effects after accounting for those nuisance factors. `TrainSel` handles this case by letting you supply an environmental design matrix E alongside the marker information.

3.1 D-optimality with environmental covariates

Extend the linear model to

$$y = E\beta_{\text{env}} + f(M)\beta_f + \epsilon,$$

with E the $n \times p$ design matrix for nuisance effects. Standard projection algebra gives that $\hat{\beta}_f$ has covariance proportional to $\left[f(M)'(I - E(E'E)^{-1}E')f(M)\right]^{-1}$. The matrix $P_E = I - E(E'E)^{-1}E'$ is the projector orthogonal to the column space of E, so we are maximizing the log-determinant of the **residualized** information matrix.

Because the design now distinguishes between, say, "row 1, col 1" and "row 3, col 4", which genotype lands in which slot **matters**. We therefore declare the set as ordered (`settypes = "OS"`).

```r
# Toy 5 x 10 row-column layout, with row x col interaction effects
E <- expand.grid(row = paste("row", 1:5, sep = "_"),
```

```
                col = paste("col", 1:10, sep = "_"))
E$row <- as.factor(E$row)
E$col <- as.factor(E$col)

DesignE <- model.matrix(~ row + col + row * col, data = E)

# Projection onto the orthogonal complement of E's column space
P <- diag(nrow(DesignE)) - DesignE %*% solve(crossprod(DesignE)) %*% t(DesignE)

dataDoptEnv <- list(FeatureMat = PC, Projection = P)

DOPTwithE <- function(soln, Data) {
  Fmat <- Data[["FeatureMat"]]
  P    <- Data[["Projection"]]
  determinant(crossprod(P %*% Fmat[soln, ]), logarithm = TRUE)$modulus
}

TSOUTDwithE <- TrainSel(
  Data       = dataDoptEnv,
  Candidates = list(1:160),
  setsizes   = c(50),
  settypes   = "OS",
  Stat       = DOPTwithE,
  control    = TSC,
  Verbose    = FALSE
)
```

```
## Maximum number of iterations reached.
```

```
# Order of BestSol_int now encodes the row/col slot each genotype occupies
TSOUTDwithE$BestSol_int
```

```
##  [1] 135  85  42  13 160  72 105  33  58  74 101  41  61 126 112  16 117 113 109
## [20]  69 142 149 124 121  17 156  19 103   4  84  81  99 154 136  18 133 146  35
## [39]  57 144 139  71  15 128 107  78 125  52  47 148
```

```
head(rownames(Wheat.M)[TSOUTDwithE$BestSol_int])
```

```
## [1] "Line135" "Line85"  "Line42"  "Line13"  "Line160" "Line72"
```

```
# Attach genotype IDs to the layout
E$GID <- rownames(Wheat.M)[TSOUTDwithE$BestSol_int]
head(E)
```

```
##     row   col     GID
## 1 row_1 col_1 Line135
## 2 row_2 col_1  Line85
## 3 row_3 col_1  Line42
## 4 row_4 col_1  Line13
## 5 row_5 col_1 Line160
## 6 row_1 col_2  Line72
```

3.2 CDmean with environmental covariates

The same idea applies to CDmean. Now the projection inside the CD matrix uses $P = V^{-1} - V^{-1}E(E'V^{-1}E)^{-1}E'V^{-1}$, i.e. the residualizer in the metric induced by V^{-1}.

```
dataCDMEANoptwithEnv <- list(G = Wheat.K[1:160, 1:160], E = DesignE, lambda = 1)

CDMEANOPTwithEnv <- function(soln, Data) {
  G      <- Data[["G"]]
  E      <- Data[["E"]]
  lambda <- Data[["lambda"]]
  Vinv   <- solve(G[soln, soln] + lambda * diag(length(soln)))
  outmat <- (G[, soln] %*%
              (Vinv - Vinv %*% E %*% solve(t(E) %*% Vinv %*% E) %*% t(E) %*% Vinv) %*%
              G[soln, ]) / G
  mean(diag(outmat[-soln, -soln]))
}

TSOUTCDwithENV <- TrainSel(
  Data       = dataCDMEANoptwithEnv,
  Candidates = list(1:160),
  setsizes   = c(50),
  settypes   = "OS",
  Stat       = CDMEANOPTwithEnv,
  control    = TSC,
  Verbose    = FALSE
)
```

```
## Convergence Achieved
##  (no improv in the last 'minitbefstop' iters).
```

```
E$GID <- rownames(Wheat.M)[TSOUTCDwithENV$BestSol_int]
head(E)
```

```
##     row   col     GID
## 1 row_1 col_1 Line138
## 2 row_2 col_1 Line135
## 3 row_3 col_1 Line101
## 4 row_4 col_1  Line51
## 5 row_5 col_1 Line160
## 6 row_1 col_2  Line74
```

3.3 Targeted CDmean with replicates

We can combine three features at once: an explicit design matrix, a target set, and replication of the same genotype across plots. The latter is enabled by declaring the set as an **ordered multi-set** (`OMS`), which allows the same genotype to appear in more than one slot.

```
dataCDMEANoptwithEnvTarget <- list(G = Wheat.K, E = DesignE,
                                   Target = 161:200, lambda = 1)

CDMEANOPTwithEnvTarget <- function(soln, Data) {
  G      <- Data[["G"]]
  E      <- Data[["E"]]
  targ   <- Data[["Target"]]
  lambda <- Data[["lambda"]]
  Vinv   <- solve(G[soln, soln] + lambda * diag(length(soln)))
  outmat <- (G[, soln] %*%
              (Vinv - Vinv %*% E %*% solve(t(E) %*% Vinv %*% E) %*% t(E) %*% Vinv) %*%
              G[soln, ]) / G
```

```
  mean(diag(outmat[targ, targ]))
}

TSOUTCDwithENVTarg <- TrainSel(
  Data       = dataCDMEANoptwithEnvTarget,
  Candidates = list(1:160),
  setsizes   = c(50),
  settypes   = "OMS",
  Stat       = CDMEANOPTwithEnvTarget,
  control    = TSC,
  Verbose    = FALSE
)
```

```
## Convergence Achieved
##  (no improv in the last 'minitbefstop' iters).
```

```
E$GID <- rownames(Wheat.M)[TSOUTCDwithENVTarg$BestSol_int]
head(E)
```

```
##     row   col     GID
## 1 row_1 col_1  Line98
## 2 row_2 col_1  Line19
## 3 row_3 col_1  Line16
## 4 row_4 col_1 Line106
## 5 row_5 col_1  Line26
## 6 row_1 col_2  Line40
```

4. Multi-environment trials

Most genomic-selection programs phenotype across multiple environments. Doing so optimally means deciding **which genotypes to evaluate in which environment** so that the resulting multi-environment prediction model generalizes well.

4.1 Setting up a multi-environment CDmean

Suppose we have three environments with the following constraints:

- **Env 1**: 30 genotypes in a 6-row × 5-column layout (ordered).
- **Env 2**: 20 genotypes in a 4-row × 5-column layout (ordered).
- **Env 3**: 50 genotypes in an unspecified homogeneous design (unordered).

We assume known (proportional) genomic and residual covariance matrices across environments:

$$V_g = \begin{pmatrix} 1.0 & 0.7 & 0.5 \\ 0.7 & 1.2 & 0.8 \\ 0.5 & 0.8 & 1.5 \end{pmatrix}, \qquad V_e = \begin{pmatrix} 1.0 & 0 & 0 \\ 0 & 1.5 & 0 \\ 0 & 0 & 1.0 \end{pmatrix}.$$

Stacking the three environments, the multi-environment G-BLUP can be written as

$$\begin{pmatrix} y_1 \\ y_2 \\ y_3 \end{pmatrix} = E\,\beta_{\text{env}} + \begin{pmatrix} Z_1 u_1 \\ Z_2 u_2 \\ Z_3 u_3 \end{pmatrix} + \begin{pmatrix} Z_1 \epsilon_1 \\ Z_2 \epsilon_2 \\ Z_3 \epsilon_3 \end{pmatrix},$$

with $\text{vec}(u) \sim \mathcal{N}(0, V_g \otimes G)$ and $\text{vec}(\epsilon) \sim \mathcal{N}(0, V_e \otimes I)$. The Kronecker products give the joint covariances we will plug into the optimizer.

```r
Vg <- matrix(c(1.0, 0.7, 0.5,
               0.7, 1.2, 0.8,
               0.5, 0.8, 1.5), 3, 3)
Ve <- matrix(c(1.0, 0,   0,
               0,   1.5, 0,
               0,   0,   1.0), 3, 3)

rownames(Vg) <- colnames(Vg) <-
rownames(Ve) <- colnames(Ve) <- paste0("E", 1:3)

G <- kronecker(Vg, Wheat.K,             make.dimnames = TRUE)
R <- kronecker(Ve, diag(nrow(Wheat.K)), make.dimnames = TRUE)

# Inspect dimensions and row labels
head(rownames(G))
```

```
## [1] "E1:Line1" "E1:Line2" "E1:Line3" "E1:Line4" "E1:Line5" "E1:Line6"
```

```r
tail(rownames(G))
```

```
## [1] "E3:Line195" "E3:Line196" "E3:Line197" "E3:Line198" "E3:Line199"
## [6] "E3:Line200"
```

```r
dim(G)
```

```
## [1] 600 600
```

Because of how `kronecker` lays out names, the candidate genotypes for Env 1, Env 2 and Env 3 occupy rows 1–160, 201–360 and 401–560 of `G`, respectively. We will sample **30** ordered slots from the first block, **20** ordered slots from the second, and **50** unordered slots from the third, with no within-environment duplicates.

4.2 Building the environmental design matrix

```r
E1 <- expand.grid(row = paste("row", 1:6, sep = "_"),
                  col = paste("col", 1:5, sep = "_"))
E2 <- expand.grid(row = paste("row", 1:4, sep = "_"),
                  col = paste("col", 1:5, sep = "_"))
E3 <- data.frame(row = paste("row", rep(1, 50), sep = "_"),
                 col = paste("col", rep(1, 50), sep = "_"))

EnvData <- data.frame(
  Env = c(rep("E1", 30), rep("E2", 20), rep("E3", 50)),
  rbind(E1, E2, E3)
)

DesignE <- model.matrix(~ Env + Env / (row + col), data = EnvData)
DesignE <- DesignE[, colSums(DesignE) > 0]
colnames(DesignE)
```

```
##  [1] "(Intercept)"    "EnvE2"          "EnvE3"          "EnvE1:rowrow_2"
##  [5] "EnvE2:rowrow_2" "EnvE1:rowrow_3" "EnvE2:rowrow_3" "EnvE1:rowrow_4"
##  [9] "EnvE2:rowrow_4" "EnvE1:rowrow_5" "EnvE1:rowrow_6" "EnvE1:colcol_2"
## [13] "EnvE2:colcol_2" "EnvE1:colcol_3" "EnvE2:colcol_3" "EnvE1:colcol_4"
## [17] "EnvE2:colcol_4" "EnvE1:colcol_5" "EnvE2:colcol_5"
```

4.3 Running the multi-environment CDmean

The objective averages the CD over the genotypes that are *neither* in the training set *nor* in the (un-targeted) target list, i.e. the remaining candidates we still want to predict well.

```
dataCDMEANoptwithEnvME <- list(G = G, R = R, E = DesignE)
Target <- c(161:200, 361:400, 561:600)  # excluded from the averaging set

CDMEANOPTwithEnvME <- function(soln, Data) {
  G    <- Data[["G"]]
  R    <- Data[["R"]]
  E    <- Data[["E"]]
  Vinv <- solve(G[soln, soln] + R[soln, soln])
  outmat <- (G[, soln] %*%
             (Vinv - Vinv %*% E %*% solve(t(E) %*% Vinv %*% E) %*% t(E) %*% Vinv) %*%
             G[soln, ]) / G
  mean(diag(outmat[-c(soln, Target), -c(soln, Target)]))
}

TSOUTCDwithENVME <- TrainSel(
  Data       = dataCDMEANoptwithEnvME,
  Candidates = list(1:160, 201:360, 401:560),
  setsizes   = c(30, 20, 50),
  settypes   = c("OS", "OS", "UOS"),
  Stat       = CDMEANOPTwithEnvME,
  control    = TSC,
  Verbose    = FALSE
)
```

```
## Maximum number of iterations reached.
```

```
# The returned vector is sized 30 + 20 + 50 = 100; we split it back per env
E1$GID <- rownames(G)[TSOUTCDwithENVME$BestSol_int[1:30]]
E2$GID <- rownames(G)[TSOUTCDwithENVME$BestSol_int[31:50]]
E3$GID <- rownames(G)[TSOUTCDwithENVME$BestSol_int[51:100]]

head(E1)
```

```
##     row   col        GID
## 1 row_1 col_1  E1:Line34
## 2 row_2 col_1  E1:Line59
## 3 row_3 col_1 E1:Line118
## 4 row_4 col_1 E1:Line159
## 5 row_5 col_1  E1:Line71
## 6 row_6 col_1  E1:Line92
```

```
head(E2)
```

```
##     row   col        GID
## 1 row_1 col_1  E2:Line80
## 2 row_2 col_1  E2:Line20
## 3 row_3 col_1 E2:Line126
## 4 row_4 col_1  E2:Line69
## 5 row_1 col_2 E2:Line106
## 6 row_2 col_2  E2:Line41
```

```
head(E3)
```

```
##     row   col       GID
## 1 row_1 col_1  E3:Line4
## 2 row_1 col_1  E3:Line6
## 3 row_1 col_1  E3:Line7
## 4 row_1 col_1  E3:Line8
## 5 row_1 col_1  E3:Line9
## 6 row_1 col_1 E3:Line11
```

4.4 Constraining the total number of unique genotypes

Breeders often have a logistic constraint on the **total** number of distinct genotypes used across all environments (seed availability, propagation capacity, etc.). Suppose we want this total to land between 80 and 90 lines.

We enforce this with a **penalty function**. If a solution violates the constraint, the penalty function returns a large negative value whose magnitude reflects *how badly* the constraint is violated; if it is satisfied, the penalty is zero and the original objective applies. The optimizer then naturally steers toward feasible regions.

```
PenaltyFunction <- function(soln) {
  soln1 <- soln[1:30]
  soln2 <- soln[31:50] - 200
  soln3 <- soln[51:100] - 400
  numuniquegeno <- length(unique(c(soln1, soln2, soln3)))
  if (numuniquegeno < 80 || numuniquegeno > 90) {
    min(numuniquegeno - 80, 90 - numuniquegeno)
  } else {
    0
  }
}
```

Wrap the CDmean objective so that infeasible solutions are penalized **before** any matrix algebra is performed:

```
CDMEANOPTwithEnvMEwithPenalty <- function(soln, Data) {
  penalty <- PenaltyFunction(soln)
  if (penalty == 0) {
    G    <- Data[["G"]]
    R    <- Data[["R"]]
    E    <- Data[["E"]]
    Vinv <- solve(G[soln, soln] + R[soln, soln])
    outmat <- (G[, soln] %*%
               (Vinv - Vinv %*% E %*% solve(t(E) %*% Vinv %*% E) %*% t(E) %*% Vinv) %*%
               G[soln, ]) / G
    mean(diag(outmat[-c(soln, Target), -c(soln, Target)]))
  } else {
    penalty
  }
}

TSOUTCDwithENVMEwithPenalty <- TrainSel(
  Data       = dataCDMEANoptwithEnvME,
  Candidates = list(1:160, 201:360, 401:560),
  setsizes   = c(30, 20, 50),
  settypes   = c("OS", "OS", "UOS"),
  Stat       = CDMEANOPTwithEnvMEwithPenalty,
```

```
  control    = TSC,
  Verbose    = FALSE
)
```

```
## Maximum number of iterations reached.
```

```
E1$GID <- rownames(G)[TSOUTCDwithENVMEwithPenalty$BestSol_int[1:30]]
E2$GID <- rownames(G)[TSOUTCDwithENVMEwithPenalty$BestSol_int[31:50]]
E3$GID <- rownames(G)[TSOUTCDwithENVMEwithPenalty$BestSol_int[51:100]]

head(E1)
```

```
##     row   col        GID
## 1 row_1 col_1  E1:Line78
## 2 row_2 col_1  E1:Line32
## 3 row_3 col_1 E1:Line120
## 4 row_4 col_1  E1:Line95
## 5 row_5 col_1  E1:Line79
## 6 row_6 col_1 E1:Line121
```

```
head(E2)
```

```
##     row   col        GID
## 1 row_1 col_1 E2:Line112
## 2 row_2 col_1 E2:Line158
## 3 row_3 col_1 E2:Line134
## 4 row_4 col_1 E2:Line102
## 5 row_1 col_2 E2:Line124
## 6 row_2 col_2  E2:Line34
```

```
head(E3)
```

```
##     row   col       GID
## 1 row_1 col_1  E3:Line3
## 2 row_1 col_1  E3:Line6
## 3 row_1 col_1  E3:Line7
## 4 row_1 col_1 E3:Line11
## 5 row_1 col_1 E3:Line14
## 6 row_1 col_1 E3:Line16
```

5. Beyond training-set design

Because `TrainSel` only requires a user-supplied scalar objective, it can be applied to many breeding problems that have nothing to do with training-set selection per se. This section illustrates four such applications.

5.1 Unsupervised marker selection (anchor markers)

Sometimes one wants to retain a *representative* subset of markers, perhaps to design a low-cost genotyping panel, or to anchor a subsequent prediction model. A natural objective is to choose the subset whose pairwise-distance structure best mimics that of the full marker set. We measure that with the **distance-correlation** statistic `bcdcor` from the `energy` package.

```
library(energy)

dataVarSel <- list(dist(Wheat.M_centered), Wheat.M_centered)
```

```r
funVarSel <- function(soln, Data) {
  bcdcor(Data[[1]], dist(Data[[2]][, soln]))
}

TSOUTVarSel <- TrainSel(
  Data       = dataVarSel,
  Candidates = list(1:ncol(Wheat.M_centered)),
  setsizes   = c(100),
  settypes   = c("UOS"),
  Stat       = funVarSel,
  control    = TSC,
  Verbose    = FALSE
)
```

```
## Maximum number of iterations reached.
```

```r
AnchorMarkers <- TSOUTVarSel$BestSol_int
```

The selected `AnchorMarkers` will be reused in the next subsection as the predictor set for a downstream prediction model.

5.2 Optimal parental proportions

We now want to **choose 3 parents** out of the 40 lines in the target set and the **proportion** of each in the resulting bulk-cross so that (i) the mean GEBV of the progeny is maximized and (ii) the inbreeding coefficient of the progeny is minimized. This is a mixed-integer problem: 3 discrete parent IDs plus 3 continuous proportions.

Step 1, Estimate marker effects on the anchor panel

```r
library(EMMREML)
mixedmodelout <- emmreml(
  y = Wheat.Y[1:160, 2],
  X = matrix(1, nrow = 160, ncol = 1),
  Z = Wheat.M_centered[1:160, AnchorMarkers],
  K = diag(length(AnchorMarkers))
)
markereffects <- mixedmodelout$uhat
```

Step 2, Compute GEBVs for the 40 prospective parents

```r
GEBVs40 <- Wheat.M_centered[161:200, AnchorMarkers] %*% markereffects
K40     <- Wheat.K[161:200, 161:200]
```

Because we know the (simulated) true values for the target set, we can take a peek at prediction accuracy. In a real breeding application this check would not be available; here it serves as a sanity diagnostic.

```r
cor(GEBVs40, Wheat.Y[161:200, 2])
```

```
##           [,1]
## [1,] 0.6073164
```

Step 3, Joint integer + continuous optimization

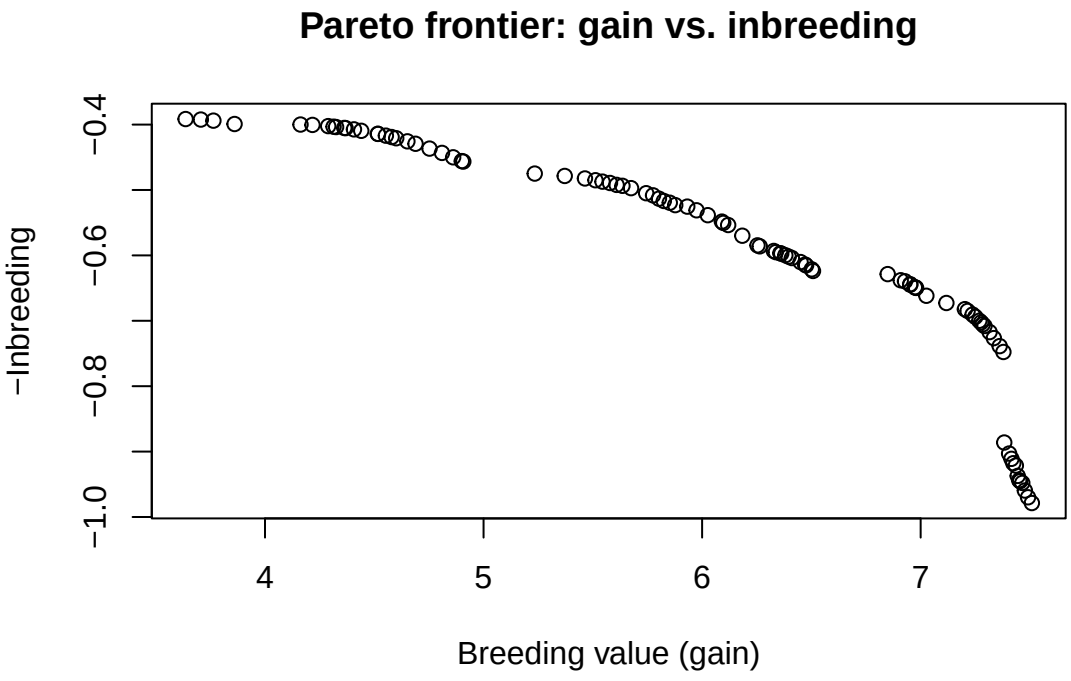
The objective returns two values (gain, –inbreeding), so this is a multi-objective run. Note the **two-list `Candidates`**: the first vector is the integer candidate IDs (`1:40`), the second is the lower/upper bounds of

the continuous variables. The `setsizes` and `settypes` arguments parallel that structure: 3 integers (`OS`) plus 3 doubles (`DBL`).

```
dataOptProp <- list(GEBVs40, K40)

funOptProp <- function(soln_int, soln_dbl, Data) {
  props <- soln_dbl / sum(soln_dbl)                          # normalize to a simplex
  BV    <- crossprod(Data[[1]][soln_int], props)             # expected gain
  Inb   <- t(props) %*% Data[[2]][soln_int, soln_int] %*% props  # expected inbreeding
  c(BV, -as.numeric(Inb))
}

TSOUTOptProp <- TrainSel(
  Data       = dataOptProp,
  Candidates = list(1:40, c(0.001, 0.999)),
  setsizes   = c(3, 3),
  settypes   = c("OS", "DBL"),
  Stat       = funOptProp,
  nStat      = 2,
  Verbose    = FALSE,
  control    = TSC
)
```

```
## Maximum number of iterations reached.
```

```
plot(t(TSOUTOptProp$BestVal),
     xlab = "Breeding value (gain)",
     ylab = "-Inbreeding",
     main = "Pareto frontier: gain vs. inbreeding")
```



5.3 Optimal genomic mating

Rather than parental *proportions* we now optimize over **mating pairs**. With 20 candidate males and 20 candidate females we enumerate all $20 \times 20 = 400$ possible crosses, and ask `TrainSel` to pick a subset of

crosses that jointly maximize expected gain and minimize expected inbreeding of progeny.

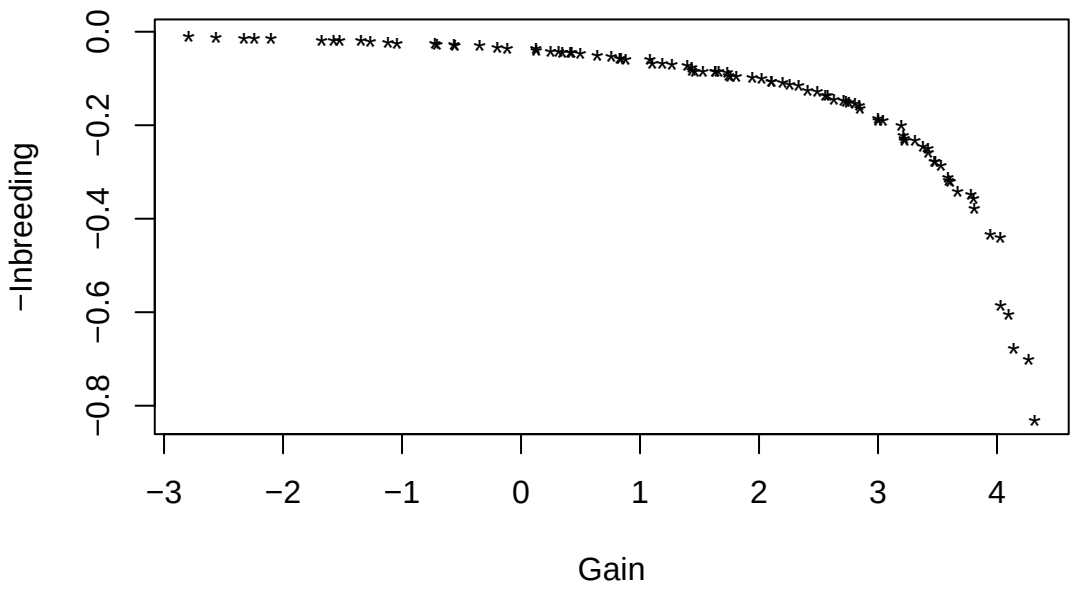
5.3.1 Two objectives: gain and inbreeding

```
library(rrBLUP)
# Candidate parents: 161:180 act as males, 181:200 as females
DFMates    <- expand.grid(rownames(Wheat.M[161:180, ]),
                          rownames(Wheat.M[181:200, ]))
M1         <- Wheat.M[161:180, AnchorMarkers]
M2         <- Wheat.M[181:200, AnchorMarkers]
M          <- rbind(M1, M2)
K          <- A.mat(M)
BVParents  <- M %*% markereffects
```

```
BV_INB <- function(soln) {
  DFMatessoln <- DFMates[soln, ]
  P1 <- factor(DFMatessoln[, 1], levels = rownames(M1))
  P2 <- factor(DFMatessoln[, 2], levels = rownames(M2))
  P1 <- model.matrix(~ P1 - 1)
  P2 <- model.matrix(~ P2 - 1)
  P  <- cbind(P1 / 2, P2 / 2)

  PKP <- mean(tcrossprod(P %*% K, P))    # inbreeding proxy
  BV  <- mean(P %*% BVParents)           # expected gain
  out <- c(BV, -as.numeric(PKP))
  names(out) <- c("Gain", "-Inb")
  out
}

# Setsize 50 with UOMS allows the same cross to be picked multiple times
TSOUTGM <- TrainSel(
  Candidates = list(1:nrow(DFMates)),
  setsizes   = 50,
  settypes   = c("UOMS"),
  Stat       = BV_INB,
  nStat      = 2,
  Verbose    = FALSE,
  control    = TSC
)
```

```
## Maximum number of iterations reached.
```

```
plot(t(TSOUTGM$BestVal), pch = "*",
     xlab = "Gain", ylab = "-Inbreeding",
     main = "Pareto frontier: gain vs. inbreeding (mating)")
```




```
# BestSol_int is now a matrix: rows = mating slots, cols = frontier solutions
dim(TSOUTGM$BestSol_int)
```

```
## [1] 50 94
```

```
# Inspect the first two frontier solutions
head(DFMates[TSOUTGM$BestSol_int[, 1], ])
```

```
##         Var1    Var2
## 43   Line163 Line183
## 43.1 Line163 Line183
## 43.2 Line163 Line183
## 43.3 Line163 Line183
## 47   Line167 Line183
## 47.1 Line167 Line183
```

```
TSOUTGM$BestVal[, 1]
```

```
## [1]  1.52912743 -0.08041398
```

```
head(DFMates[TSOUTGM$BestSol_int[, 2], ])
```

```
##         Var1    Var2
## 24   Line164 Line182
## 54   Line174 Line183
## 55   Line175 Line183
## 58   Line178 Line183
## 58.1 Line178 Line183
## 59   Line179 Line183
```

```
TSOUTGM$BestVal[, 2]
```

```
## [1] -0.70493070 -0.02248174
```

5.3.2 Three objectives: gain, cross-variance, inbreeding

When the optimization variable is a list of *crosses* (rather than parental proportions), an extra criterion becomes available: the **within-family variance** of each cross. The package ships two helpers, `calculatecrossvalueM1` and `calculatecrossvalueM2`, that estimate this variance from parental marker data and marker effects. Adding it gives a three-way trade-off between gain, useful within-family variance, and inbreeding.

```
BV_VAR_INB <- function(soln) {
  DFMatessoln <- DFMates[soln, ]
  P1 <- factor(DFMatessoln[, 1], levels = rownames(M1))
  P2 <- factor(DFMatessoln[, 2], levels = rownames(M2))
  P1 <- model.matrix(~ P1 - 1)
  P2 <- model.matrix(~ P2 - 1)
  P  <- cbind(P1 / 2, P2 / 2)

  BVmatessoln <- mean(P %*% BVParents)           # gain
  varI <- mean(sapply(seq_len(nrow(DFMatessoln)), function(i) {
    p1 <- DFMatessoln[i, 1]
    p2 <- DFMatessoln[i, 2]
    m1 <- M1[p1, ]
    m2 <- M2[p2, ]
    calculatecrossvalueM1(round(m1), round(m2), markereffects)
    # Alternative with a marker map:
    # calculatecrossvalueM2(round(m1), round(m2), markereffects, markermap)
  }))
  PKP <- mean(tcrossprod(P %*% K, P))            # inbreeding

  out <- c(BVmatessoln, varI, -as.numeric(PKP))
  names(out) <- c("Gain", "Var", "-Inb")
  out
}
```

A properly sampled Pareto surface in three dimensions would require more iterations and a larger population than the demo permits; here we run a small toy problem just to illustrate the mechanics.

```
TSOUT <- TrainSel(
  Candidates = list(1:nrow(DFMates)),
  setsizes   = 5,
  settypes   = c("UOMS"),
  Stat       = BV_VAR_INB,
  nStat      = 3,
  control    = TSC,
  Verbose    = FALSE
)
```

```
## Maximum number of iterations reached.
```

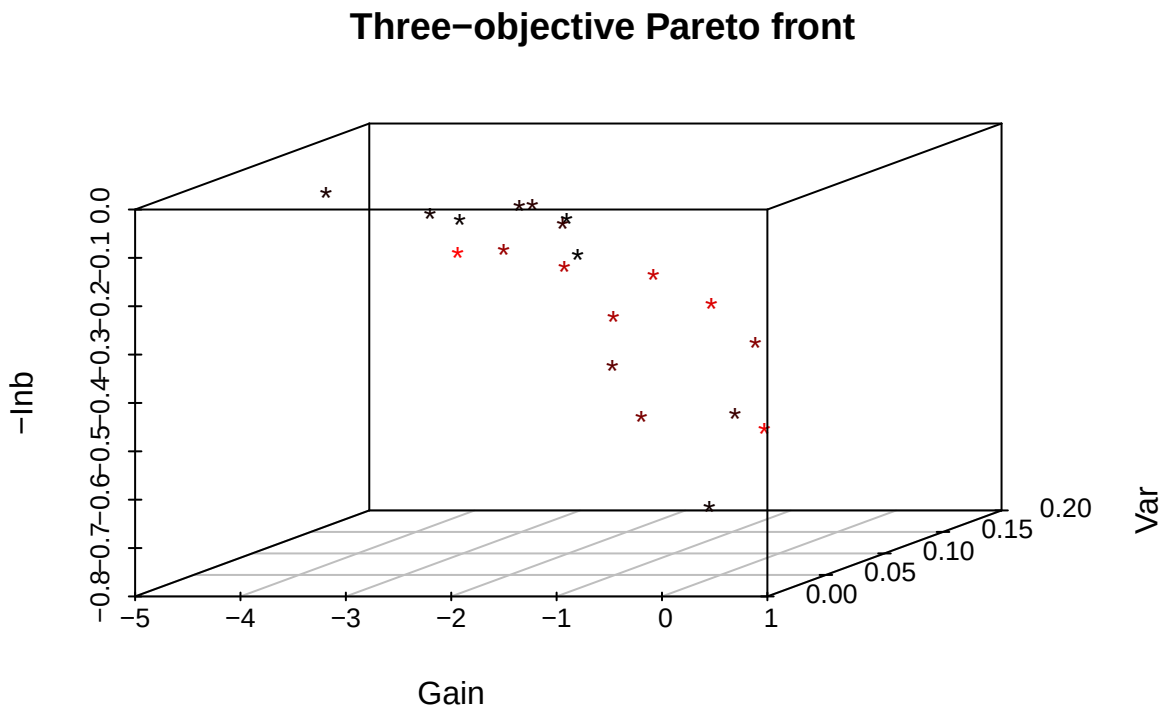
```
TSOUT$BestVal[, 1:5]      # objective values of the first 5 frontier points
```

```
##            [,1]        [,2]        [,3]       [,4]         [,5]
## [1,] -3.5416843 -2.52645543 -0.55875557 -3.3475819  0.943745537
## [2,]  0.1205862  0.10551405  0.03255927  0.1284422  0.002447344
## [3,] -0.1124522 -0.08070118 -0.45288044 -0.1313102 -0.450822575
```

```
ncol(TSOUT$BestVal)       # total number of non-dominated solutions found
```

```
## [1] 20
```

```
library(scatterplot3d)
scatterplot3d(t(TSOUT$BestVal), pch = "*", highlight.3d = TRUE,
              xlab = "Gain", ylab = "Var", zlab = "-Inb",
              main = "Three-objective Pareto front")
```




```
# A couple of representative solutions
head(DFMates[TSOUT$BestSol_int[, 1], ])
```

```
##        Var1    Var2
## 5   Line165 Line181
## 115 Line175 Line186
## 203 Line163 Line191
## 258 Line178 Line193
## 396 Line176 Line200
```

```
TSOUT$BestVal[, 1]
```

```
## [1] -3.5416843  0.1205862 -0.1124522
```

```
head(DFMates[TSOUT$BestSol_int[, 10], ])
```

```
##        Var1    Var2
## 94  Line174 Line185
## 227 Line167 Line192
## 334 Line174 Line197
## 342 Line162 Line198
## 357 Line177 Line198
```

```
TSOUT$BestVal[, 10]
```

```
## [1]  0.5296128  0.0322138 -0.2998751
```

5.4 General-purpose function optimization

Although it was designed for breeding-flavored combinatorial problems, `TrainSel` can also optimize plain real-valued functions by declaring the input as `DBL` with given bounds.

5.4.1 One continuous input

```
fobj  <- function(x) (x^2 + x) * cos(x)
lbound <- -10
ubound <-  10
xgrid  <- seq(lbound, ubound, length = 100)
```

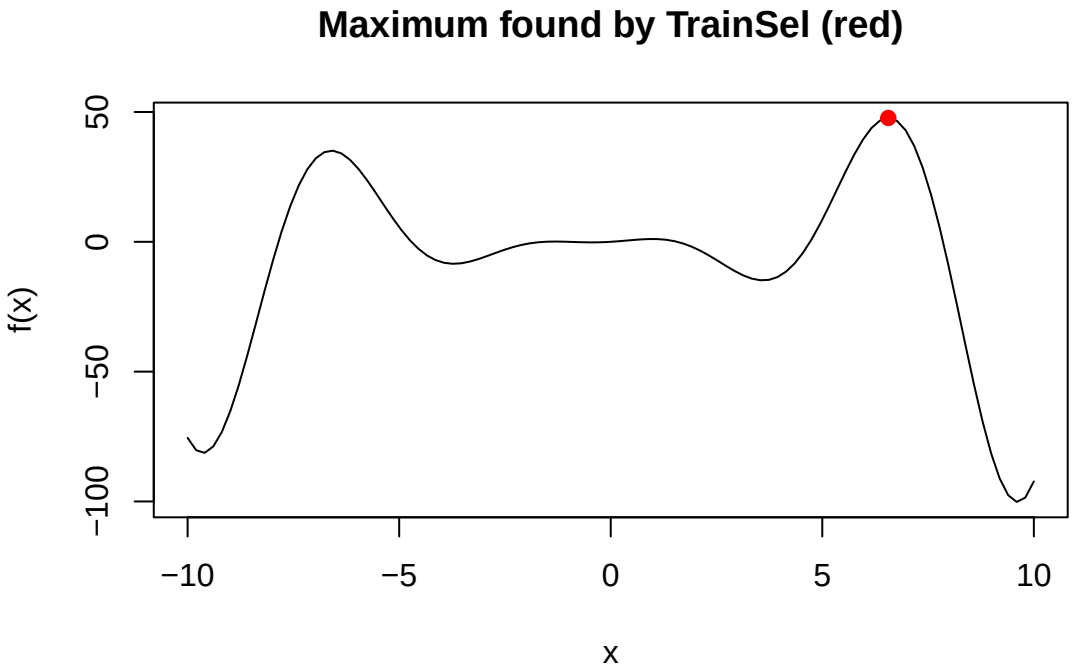
```
TSout <- TrainSel(
  Candidates = list(c(lbound, ubound)),
  setsizes   = 1,
  settypes   = c("DBL"),
  Stat       = fobj,
  Verbose    = FALSE,
  control    = TSC
)
```

```
## Convergence Achieved
##  (no improv in the last 'minitbefstop' iters).
```

```
TSout$BestSol_DBL
```

```
## [1] 6.560539
```

```
plot(xgrid, fobj(xgrid), type = "l",
     xlab = "x", ylab = "f(x)",
     main = "Maximum found by TrainSel (red)")
points(TSout$BestSol_DBL, TSout$BestVal, col = "red", pch = 19)
```



5.4.2 Two continuous inputs (Rastrigin function)

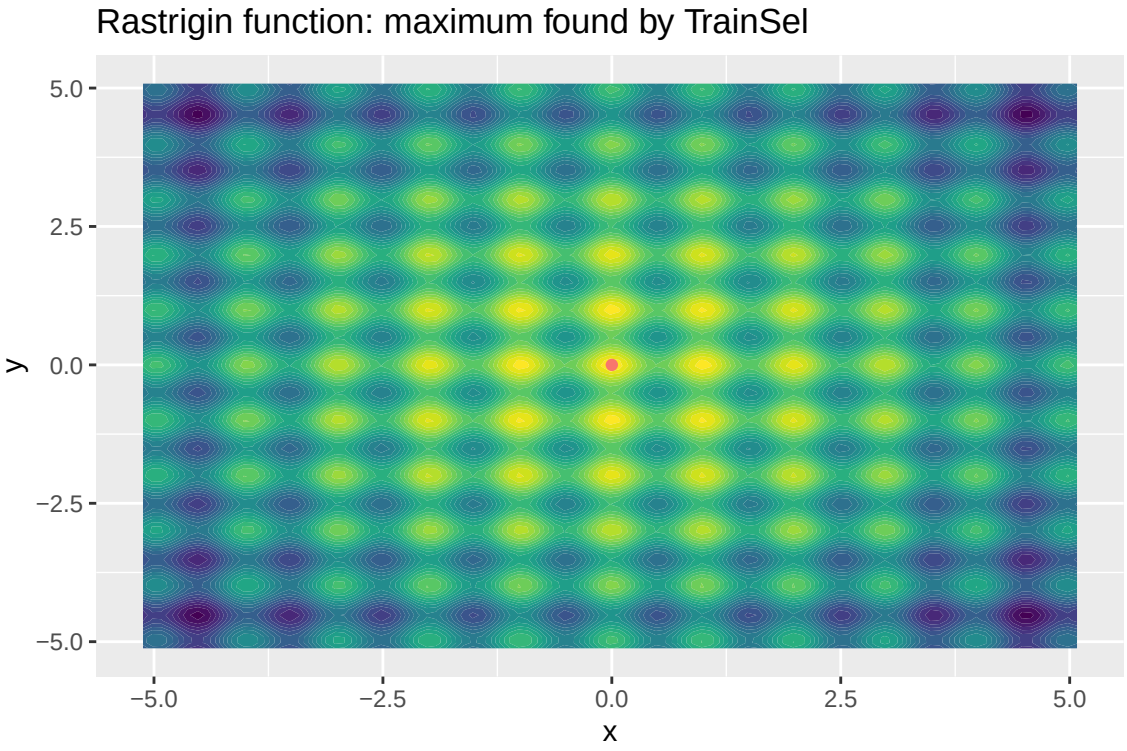
The Rastrigin function is a classical benchmark with many local optima. Note that we negate it inside the objective because `TrainSel` *maximizes*.

```
Rastrigin <- function(x) {
  x1 <- x[1]; x2 <- x[2]
  -(20 + x1^2 + x2^2 - 10 * (cos(2 * pi * x1) + cos(2 * pi * x2)))
}
```

```
TSout2 <- TrainSel(
  Candidates = list(c(-10, 5.12), c(-5.12, 10)),
  setsizes   = 2,
  settypes   = c("DBL", "DBL"),
  Stat       = Rastrigin,
  Verbose    = FALSE,
  control    = TSC
)
```

```
## Convergence Achieved
##  (no improv in the last 'minitbefstop' iters).
```

```
x1best <- TSout2$BestSol_DBL[1]
x2best <- TSout2$BestSol_DBL[2]
xbest  <- data.frame(x = x1best, y = x2best)

xg <- seq(-5.12, 5.12, by = 0.1)
x12 <- expand.grid(xg, xg)
f   <- apply(x12, 1, Rastrigin)

plotdata <- cbind(x12, f)
colnames(plotdata) <- c("x", "y", "z")

library(ggplot2)
ggplot(plotdata, aes(x, y)) +
  stat_contour_filled(aes(z = z), bins = 30) +
  geom_point(data = xbest, aes(color = "best")) +
  theme(legend.position = "none") +
  labs(title = "Rastrigin function: maximum found by TrainSel")
```



5.4.3 A vector-valued (multi-objective) toy

```
library(devtools)
library(BBmisc)

fn <- function(x) {
  c(x[1]^2 - x[2]^2,
    sin(x[2]^2 - x[1]^2),
    x[1]^2 + x[2]^2)
}

lower <- c(0, 0, 0)
upper <- c(1, 1, 1)
fn(c(0, 1))
```

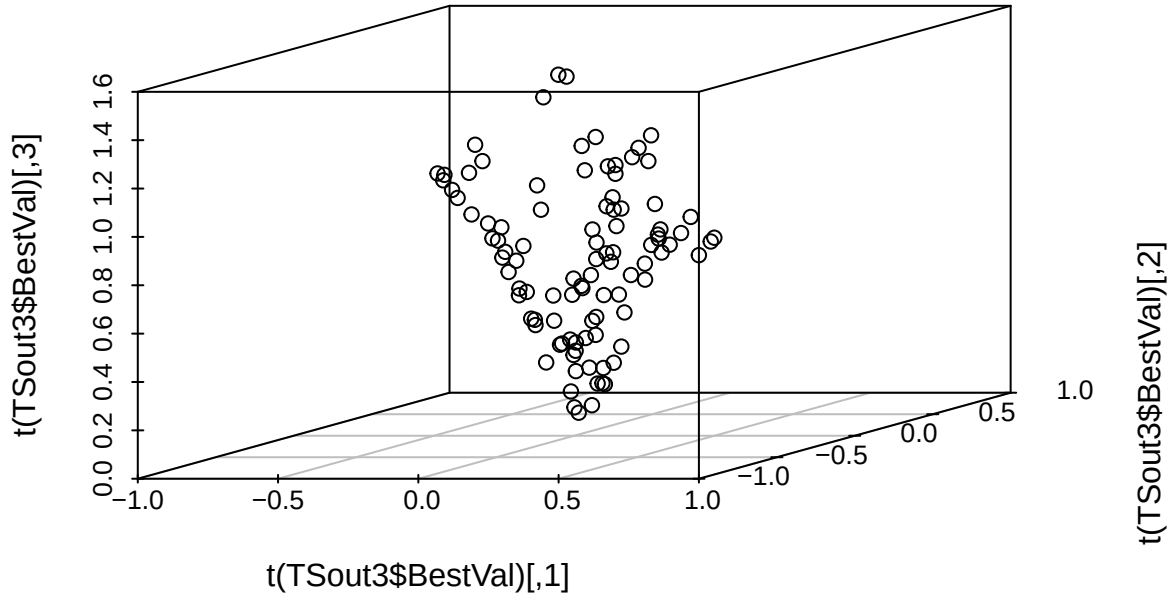
```
## [1] -1.000000  0.841471  1.000000
```

```
tsControl <- TSC
tsControl$niterations <- 10  # toy run, bump this up in real applications

TSout3 <- TrainSel(
  Candidates = list(c(lower[1], upper[1]),
                    c(lower[2], upper[2]),
                    c(lower[3], upper[3])),
  setsizes   = 2,
  settypes   = c("DBL", "DBL"),
  Stat       = fn,
  nStat      = 3,
  control    = tsControl,
  Verbose    = FALSE
```

```
)
```

```
## Maximum number of iterations reached.
```

```
scatterplot3d::scatterplot3d(t(TSout3$BestVal))
```



6. Hyperparameters, complexity, and computation time

The default presets in `SetControlDefault` already produce reasonable results for most problems. This section explains what those presets control, when to override them, and how to estimate how long an optimization will take.

6.1 The `SetControlDefault` settings

`SetControlDefault` builds a `TrainSel_Control` object from two arguments:

- **`size`**, `"demo"`, `"small"`, `"medium"`, `"large"`. Bigger sizes use more iterations and a bigger search population. Only `"demo"` works without a license key.
- **`complexity`**, `"low_complexity"` (genetic algorithm only, fast convergence but more easily trapped in local maxima) or `"high_complexity"` (islands + GA + simulated annealing, slower convergence but better global exploration). For multi-objective problems, set this to `"low_complexity"`.

The toy example below demonstrates the difference. We construct 200 values (100 negative, 100 positive) and ask `TrainSel` to pick 50 of them that maximize the **absolute value** of their sum. There are two competing optima: selecting only the largest positives (global maximum) or only the smallest negatives (local maximum just slightly worse).

```
candidates <- 1:200
size       <- 100
values     <- c(-((100:1) / 100)^2,   # first 100 negative
                ((3:102) / 100)^2)    # last 100 positive

DataComp <- list(values = values)

head(values); tail(values)
```

```
## [1] -1.0000 -0.9801 -0.9604 -0.9409 -0.9216 -0.9025
```

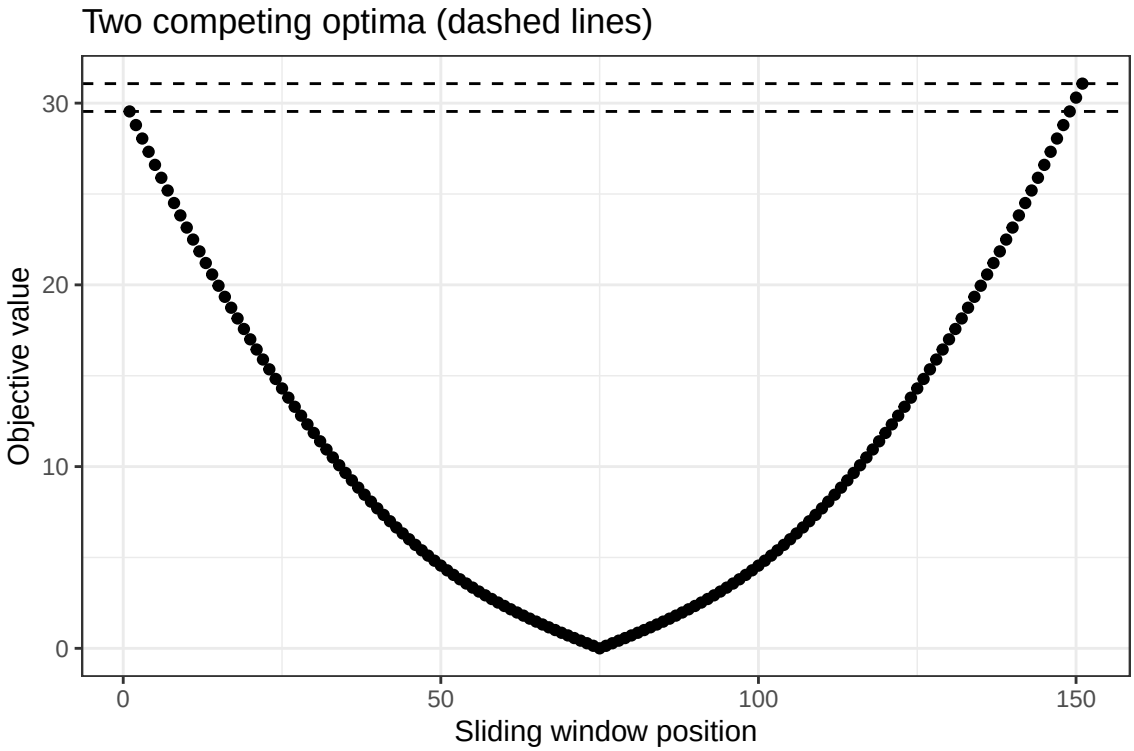
```
## [1] 0.9409 0.9604 0.9801 1.0000 1.0201 1.0404
```

```
TestEval <- function(soln, DataComp) {
  values <- DataComp[["values"]]
  abs(sum(values[soln]))
}

knownTrueBest <- abs(sum(values[151:200]))
LocalMaximum <- abs(sum(values[1:50]))
knownTrueBest; LocalMaximum
```

```
## [1] 31.0725
```

```
## [1] 29.5425
```

```
# Sliding-window plot: shows the two competing optima clearly
library(ggplot2)
selected <- list(); value <- c()
for (i in 1:151) {
  selected[[i]] <- (1:50) + i - 1
  value <- c(value, TestEval(selected[[i]], DataComp))
}
plotdf <- data.frame(Stat_value = value, Sliding_window = seq_along(value))

ggplot(plotdf) +
  theme_bw() +
  geom_point(aes(x = Sliding_window, y = Stat_value)) +
  geom_hline(yintercept = knownTrueBest, linetype = "dashed") +
  geom_hline(yintercept = LocalMaximum,  linetype = "dashed") +
  labs(x = "Sliding window position",
       y = "Objective value",
       title = "Two competing optima (dashed lines)")
```



Low vs. high complexity

The two blocks below would normally be run with a non-demo size; they require a license key, so we mark them `eval = FALSE` and load pre-computed results afterwards. With `"low_complexity"`, only the genetic algorithm is used; with `"high_complexity"`, an islands phase followed by a combined GA + simulated annealing phase is run, which is much more robust against local optima.

```
# Requires a license key, not run by default.
tsLowComp <- SetControlDefault(size = "small",
                               complexity = "low_complexity",
                               verbose = FALSE)
tsLowComp$progress <- FALSE

results_low_comp <- c()
for (i in 1:10) {
  set.seed(i)
  TSOUTC <- TrainSel(
    Data       = DataComp,
    Candidates = list(1:200),
    setsizes   = c(50),
    settypes   = "UOS",
    Stat       = TestEval,
    control    = tsLowComp,
    Verbose    = FALSE,
    Username   = NULL,
    Password   = NULL
  )
  results_low_comp <- c(results_low_comp, TSOUTC$BestVal)
}
```

```
# Requires a license key, not run by default.
tsHighComp <- SetControlDefault(size = "small",
                                complexity = "high_complexity",
                                verbose = FALSE)
tsHighComp$progress <- FALSE

results_high_comp <- c()
for (i in 1:10) {
  set.seed(i)
  TSOUTC <- TrainSel(
    Data       = DataComp,
    Candidates = list(1:200),
    setsizes   = c(50),
    settypes   = "UOS",
    Stat       = TestEval,
    control    = tsHighComp,
    Verbose    = FALSE,
    Username   = NULL,
    Password   = NULL
  )
  results_high_comp <- c(results_high_comp, TSOUTC$BestVal)
}
```

```
# The two chunks above (`low-comp`, `high-comp`) require a license key and
# therefore are not evaluated when the vignette is built. Their results are
# saved in `non_demo_results.RData`, which we load here if available. If the
# file is missing, we skip the plot rather than halt the build.
if (file.exists("non_demo_results.RData")) {
  load("non_demo_results.RData")

  plotdf <- data.frame(
    Stat_value         = c(results_low_comp, results_high_comp),
    Control_complexity = c(rep("low_complexity",  10),
                           rep("high_complexity", 10))
  )

  ggplot(plotdf) +
    theme_bw() +
    geom_point(aes(x = Control_complexity, y = Stat_value,
                   fill = Control_complexity),
               position = position_jitter(w = 0.3, h = 0),
               size = 3, shape = 21) +
    geom_hline(yintercept = knownTrueBest, linetype = "dashed") +
    geom_hline(yintercept = LocalMaximum,  linetype = "dashed")
} else {
  message("non_demo_results.RData not found - skipping the complexity ",
          "comparison plot. Reproducing these results requires a valid ",
          "license key (see Section 7).")
}
```

In our 10-run benchmark, `"high_complexity"` reached the global optimum in **every** run, whereas `"low_complexity"` got stuck at the local maximum in roughly 40% of the runs. The price is computational time: `"high_complexity"` performs many more objective-function evaluations.

6.2 Estimating computation time

`TimeEstimation` runs the user-supplied objective function a handful of times to time it, then multiplies by the number of evaluations the optimizer would have to make. The estimate is accurate when the objective is the bottleneck, and is an underestimate when the objective is so fast that internal `TrainSel` overhead dominates.

```
TimeEval <- function(soln, DataTime) {
  sleep <- DataTime[["sleep"]]
  Sys.sleep(sleep)        # simulate computation time
  1
}

TimeControl <- SetControlDefault(size = "demo",
                                 complexity = "low_complexity",
                                 verbose = FALSE)
TimeControl$npop        <- 4
TimeControl$nelite      <- 2
TimeControl$niterations <- 8

CandidatesTime <- list(1:200)
setsizesTime   <- c(50)
settypesTime   <- "UOS"
```

Slow objective, estimate matters

```
DataTime <- list(sleep = 1)  # 1 second per call

Time_resuls <- TimeEstimation(
  Stat       = TimeEval,
  Data       = DataTime,
  Candidates = CandidatesTime,
  setsizes   = setsizesTime,
  settypes   = settypesTime,
  control    = TimeControl,
  n_average  = 5,
  verbose    = FALSE
)

Time_resuls$time_Stat_vector    # per-call timings (s)
```

```
## [1] 1.005174 1.010188 1.001150 1.005227 1.003021
```

```
Time_resuls$eval_total          # total number of Stat() calls expected
```

```
## [1] 20
```

```
Time_resuls$time_total_seconds  # predicted total runtime (s)
```

```
## [1] 20.09904
```

```
# Compare with actual runtime
startTime <- Sys.time()
TSOUTT <- TrainSel(
  Data       = DataTime,
  Candidates = CandidatesTime,
  setsizes   = setsizesTime,
```

```
  settypes   = settypesTime,
  Stat       = TimeEval,
  control    = TimeControl,
  Verbose    = FALSE
)
```

```
## Maximum number of iterations reached.
```

```
endTime <- Sys.time()
endTime - startTime
```

```
## Time difference of 20.59758 secs
```

The estimate is close to the realized time. When the objective dominates, `TimeEstimation` is a useful planning tool, for instance, to decide whether to upgrade from `"demo"` to `"large"`, or whether your hardware can run the full pipeline overnight.

Fast objective, estimate is loose

```
DataTime <- list(sleep = 0)

Time_resuls <- TimeEstimation(
  Stat       = TimeEval,
  Data       = DataTime,
  Candidates = CandidatesTime,
  setsizes   = setsizesTime,
  settypes   = settypesTime,
  control    = TimeControl,
  n_average  = 10,
  verbose    = FALSE
)
Time_resuls$time_total_seconds
```

```
## [1] 0.0001616478
```

```
startTime <- Sys.time()
TSOUTT <- TrainSel(
  Data       = DataTime,
  Candidates = CandidatesTime,
  setsizes   = setsizesTime,
  settypes   = settypesTime,
  Stat       = TimeEval,
  control    = TimeControl,
  Verbose    = FALSE
)
```

```
## Maximum number of iterations reached.
```

```
endTime <- Sys.time()
endTime - startTime
```

```
## Time difference of 0.5095279 secs
```

Here the prediction is off by orders of magnitude because the *internal* `TrainSel` overhead, not the objective, dominates. That is fine: when the objective is essentially instantaneous, the absolute runtime is short anyway and the imprecision of the estimate is irrelevant in practice.

6.3 A tuning recipe

Putting the pieces together, we recommend the following workflow whenever you tackle an unfamiliar problem:

1. **Time-estimate the upper bound.** Run `TimeEstimation` with `size = "large"` and `complexity = "high_complexity"`.
   - If the predicted time is affordable, use those settings.
   - If not, drop to `"low_complexity"` and dial `size` down until the estimate is acceptable.
2. **Run the optimizer once with the chosen settings.**
3. **Inspect `length(maxvec)` versus `niterations`.**
   - Convergence reached very late (70% of `niterations`): keep settings.
   - Convergence reached around 30–70%: stay at the same `size`, escalate to `"high_complexity"`.
   - Convergence reached before 30%: reduce `size` and escalate to `"high_complexity"`.
   - Convergence not reached: increase `size`; if already `"large"`, edit `niterations` directly in the control object.

The decision tree in Figure 1 summarizes this recipe.

(Decision-tree figure `HyperparameterTuning.png` is referenced here but was not bundled with this build.)

A worked example follows. We pretend our real objective takes one second per call and walk through the tuning loop.

```
DataTime <- list(sleep = 1)

# 1.1, Try the largest, most robust preset first
exampleCNTRL <- SetControlDefault(size = "large",
                                  complexity = "high_complexity",
                                  verbose = FALSE)

Time_resuls <- TimeEstimation(
  Stat       = TimeEval,
  Data       = DataTime,
  Candidates = CandidatesTime,
  setsizes   = setsizesTime,
  settypes   = settypesTime,
  control    = exampleCNTRL,
  n_average  = 5,
  verbose    = FALSE
)
Time_resuls$time_total_seconds / 3600   # in hours, way too long here
```

```
## [1] 185.8791
```

```
# 1.2, Step down to demo size + low complexity
exampleCNTRL <- SetControlDefault(size = "demo",
                                  complexity = "low_complexity",
                                  verbose = FALSE)
Time_resuls <- TimeEstimation(
  Stat       = TimeEval,
  Data       = DataTime,
  Candidates = CandidatesTime,
  setsizes   = setsizesTime,
  settypes   = settypesTime,
  control    = exampleCNTRL,
  n_average  = 5,
  verbose    = FALSE
```

```
)
Time_resuls$time_total_seconds / 3600
```

```
## [1] 2.677527
```

```
# 1.3, Still too long for a vignette: tighten further for illustration only
exampleCNTRL$npop          <- 4
exampleCNTRL$nelite        <- 2
exampleCNTRL$niterations   <- 8
exampleCNTRL$minitbefstop  <- 1   # demo only, do NOT do this in real runs

Time_resuls <- TimeEstimation(
  Stat       = TimeEval,
  Data       = DataTime,
  Candidates = CandidatesTime,
  setsizes   = setsizesTime,
  settypes   = settypesTime,
  control    = exampleCNTRL,
  n_average  = 5,
  verbose    = FALSE
)
Time_resuls$time_total_seconds
```

```
## [1] 20.07931
```

```
# 2, Run optimization with the chosen settings
startTime <- Sys.time()
TSOUTT <- TrainSel(
  Data       = DataTime,
  Candidates = CandidatesTime,
  setsizes   = setsizesTime,
  settypes   = settypesTime,
  Stat       = TimeEval,
  control    = exampleCNTRL,
  Verbose    = FALSE
)
```

```
## Convergence Achieved
##  (no improv in the last 'minitbefstop' iters).
```

```
endTime <- Sys.time()
endTime - startTime
```

```
## Time difference of 10.54991 secs
```

```
# 2.1, Inspect convergence
length(TSOUTT$maxvec)        # iterations actually run
```

```
## [1] 3
```

```
exampleCNTRL$niterations     # iteration cap
```

```
## [1] 8
```

7. License key

This vignette runs entirely under the **demo** preset, which is limited to training-set sizes of up to 100 integers and up to 3 continuous variables. Larger problems require a license key.

```
# The following call exceeds demo limits (setsize > 100) and will error
# unless a valid license is supplied.
TSOUTCD <- TrainSel(
  Data       = dataCDMEANopt,
  Candidates = list(1:160),
  setsizes   = c(120),
  settypes   = "UOS",
  Stat       = CDMEANOPT,
  control    = TSC,
  Verbose    = FALSE
)

# Free license keys are available for universities and non-profit institutions.
# For commercial use, payment is required. Contact: j.isidro@upm.es
#
# Once a license has been issued, the username and password are passed
# directly to TrainSel():
username <- 0   # replace with your real username
password <- 0   # replace with your real password

TSOUTCD <- TrainSel(
  Data       = dataCDMEANopt,
  Candidates = list(1:160),
  setsizes   = c(120),
  settypes   = "UOS",
  Stat       = CDMEANOPT,
  control    = TSC,
  Username   = username,
  Password   = password,
  Verbose    = FALSE
)
```

Appendix, Summary of changes from the previous version

The major edits relative to `TrainSelUsage.Rmd` are:

1. **New abstract and structured introduction.** Section 1 now opens with a single explicit account of what `TrainSel` does, the canonical shape of every call, and a glossary table for the six `settypes` codes that recur throughout. The wheat dataset is described once, up front.
2. **Reorganized section hierarchy.** Three numbered top-level sections (single environment, multi-environment, beyond-training-set) replace the previously flat structure. Targeted vs. un-targeted criteria are clearly contrasted before any code appears.
3. **Pedagogical framing of each criterion.** Every objective function now has a short paragraph explaining *what it measures* and *when a breeder would reach for it*, in addition to the mathematical definition.
4. **Tightened math and prose.** Equations have been cleaned up (consistent notation, fixed stray operators in the multi-environment model), typos such as "infinitismall" → "infinitesimal" have been corrected, and long runs of academic prose have been broken into short, signposted paragraphs.
5. **Improved code style without changing logic.** Every code chunk has been indented and aligned

consistently, named (so cross-referencing is easier), and lightly commented where the *why* is non-obvious. **No** computational step has been altered.

6. **Output interpretation.** After most code chunks a short comment explains how to read the result, `BestSol_int`, `BestVal`, `maxvec`, Pareto matrices, etc.
7. **Multi-environment narrative.** Section 4 now lays out the scenario (environment sizes, ordered vs. unordered slots, covariance matrices) before the code, so the reader knows what to look for.
8. **Reordered final sections.** Hyperparameter tuning, time estimation, and the tuning recipe were consolidated into a single Section 6 with a clear decision-tree workflow. The license-key note moved to its own short Section 7.
9. **Front-matter improvements.** YAML now uses block-style author lists, a `toc_depth`, and a chunk-level cache default so that re-knitting is fast during iterative editing.

No function signatures, argument names, datasets, or numeric results were invented; the package API exposed in `TrainSelUsage_v2.Rmd` matches the `NAMESPACE` and `man/` documentation shipped with `TrainSel` v3.0.